

Полное описание проекта

Оптимизация графика работы отделений банка

Совкомбанк Технологии × Большая Математическая Мастерская, АГУ, лето 2026

Ситуация

Банковское отделение — это система массового обслуживания. Клиенты приходят, встают в очередь, обслуживаются у одного из нескольких окон и уходят. Поток клиентов неравномерный: утром — наплыв (люди по дороге на работу), в обед — ещё один пик, к вечеру — третий (после работы). В среду тихо, в понедельник и пятницу — аншлаг. В субботу — сокращённый день. В воскресенье — закрыто.

Сейчас расписание сотрудников составляется вручную руководителем: «все выходят на 9:00, обед по очереди». Это приводит к двум проблемам: в часы пик клиенты ждут по 20–30 минут, а в часы затишья сотрудники сидят без работы.

Проблема

Составить расписание «на глаз» невозможно сделать оптимально, потому что:

- **Поток клиентов случайный.** Даже в одно и то же время в разные дни приходит разное число людей. Нужна вероятностная модель.
- **Операции разные по длительности.** Перевод занимает 5 минут, а оформление кредита — 40. Нужно учитывать смесь типов обслуживания.
- **У сотрудников разные компетенции.** Не каждый может оформить ипотеку — только senior. Назначить всех senior на утро — и вечером некому будет работать.
- **Есть жёсткие ограничения.** Не больше 40 часов в неделю, не больше 9 часов в день, обязательный обеденный перерыв, минимальное число открытых окон.
- **Есть бюджет.** Senior стоит дороже junior. Нужен баланс между качеством обслуживания и затратами.

Цель проекта

По историческим данным о потоке клиентов в отделения банка:

1. Построить математическую модель очереди
2. Разработать алгоритм составления оптимального расписания сотрудников
3. Минимизировать среднее время ожидания клиента при заданных ограничениях на бюджет (ФОТ)

Данные

Участникам предоставляется синтетический датасет, включающий:

- **client_arrivals.csv** — ~6 400 визитов клиентов за 4 недели в 3 отделениях: дата, время прибытия, тип операции, фактическое время обслуживания
- **operations.csv** — справочник из 6 типов операций с нормативами времени и требуемой

квалификацией

- **employees.csv** — 22 сотрудника с грейдами (junior/middle/senior), навыками и стоимостью часа
- **branches.csv** — 3 отделения разного размера (флагманское, стандартное $\times 2$)
- **hourly_load.csv** — агрегированная нагрузка по часам (для удобства)
- **generate_dataset.py** — генератор для создания дополнительных данных с другими параметрами

Модель потока: пуассоновский процесс с переменной интенсивностью (три гауссовых пика в течение дня), коэффициенты по дням недели, случайный шум $\pm 15\%$.

Что нужно сделать

Минимум (must have)

1. **Разведочный анализ данных.** Визуализировать поток клиентов: по часам, по дням недели, по типам операций. Найти часы пиковой и минимальной нагрузки.
2. **Модель очереди.** Реализовать симуляцию системы массового обслуживания: клиенты приходят $\rightarrow$ встают в очередь $\rightarrow$ обслуживаются у свободного окна $\rightarrow$ уходят. Для заданного числа открытых окон подсчитать: среднее время ожидания, максимальное время ожидания, долю клиентов, ждавших больше 15 минут.
3. **Оптимизация числа окон по часам.** Для каждого часа каждого дня определить, сколько окон нужно открыть, чтобы среднее ожидание не превышало порог (например, 10 минут).
4. **Составление расписания.** Распределить сотрудников по сменам так, чтобы в каждый час было открыто нужное число окон, с учётом ограничений (40 ч/нед, обед, навыки).

Продвинутый уровень (nice to have)

- **Формула Эрланга.** Сравнить результаты симуляции с аналитическим решением М/М/с (формула Эрланга С). Когда формула даёт хорошее приближение, а когда нет?
- **Оптимизация по бюджету.** Фиксирован бюджет на ФОТ. Как распределить junior/middle/senior, чтобы максимизировать качество обслуживания?
- **Учёт типов операций.** Отдельные окна для быстрых и долгих операций vs единая очередь. Что лучше?
- **Робастность.** Насколько устойчиво расписание к случайным всплескам? Что если в понедельник пришло на 30% больше, чем обычно?

Математический аппарат

Задача покрывает несколько областей, но ни одна не требует продвинутой математики:

- **Пуассоновский процесс.** Модель случайного потока событий. Единственный параметр — интенсивность λ (среднее число клиентов в час). Нужно уметь оценить λ по данным.
- **Теория массового обслуживания.** Модель М/М/с: с обслуживающих каналов, пуассоновский вход, экспоненциальное время обслуживания. Формула Эрланга С даёт вероятность ожидания и среднее время в очереди аналитически.
- **Дискретно-событийная симуляция.** Альтернатива формулам: моделируем каждого клиента, каждое окно, каждую минуту. Проще реализовать, проще расширять.
- **Оптимизация с ограничениями.** Составление расписания — это задача целочисленного программирования (или жадный алгоритм, или метаэвристика). Ограничения: часы, навыки, бюджет.

Подсказки для старта

1. Начните с визуализации. Постройте heatmap: ось X — час дня, ось Y — день недели, цвет — число клиентов. Сразу увидите паттерн.
2. Реализуйте простейшую симуляцию: одно окно, один тип клиентов. Посчитайте среднее ожидание. Потом добавляйте окна.
3. Сравните с формулой Эрланга C — если совпадает, значит симуляция работает.
4. Для расписания начните с жадного алгоритма: для каждого часа определите нужное число окон, потом «раскладывайте» сотрудников по сменам.
5. Не пытайтесь оптимизировать всё сразу. Сначала — одно отделение, один день.

Критерии оценки

Критерий	Вес
Качество модели очереди (совпадение с теорией, корректность симуляции)	30%
Качество расписания (среднее время ожидания при данном бюджете)	25%
Математическая глубина (анализ, обоснование, сравнение подходов)	20%
Визуализация и презентация	15%
Оригинальность подхода	10%

Результат

По итогам работы команда представляет:

1. **Код** — скрипты/ноутбуки с документацией и инструкцией по запуску
2. **Отчёт** — описание модели, экспериментов, результатов (5–10 страниц)
3. **Визуализация** — графики нагрузки, времени ожидания, итоговое расписание
4. **Презентация** — защита решения (10 мин + 5 мин вопросы)

Рекомендуемый стек

- Python 3.10+
- pandas, numpy — работа с данными
- matplotlib / plotly — визуализация
- simpy (опционально) — библиотека дискретно-событийной симуляции
- scipy.optimize (опционально) — оптимизация
- Jupyter Notebook — для исследовательской части